

Experiment ‘p_14_4_merge_sort_lexicographic_seq’ Results

June 6, 2026

Experiment outcome: FAILURE_VERIFY

Bad responses: 1

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 3

Hard fails (resolution): 1

Soft fails (resolution): 0

Verification attempts: 2

Problem Specification

Problem name: p_14_4_merge_sort_lexicographic_seq

Natural language statement: Write a method that implements the merge sort algorithm to sort an array of strings in lexicographic order.

Method signature: p_14_4_merge_sort_lexicographic_seq(inputs: seq<string>) returns (sorted: seq<string>)

Ensures

- forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
- multiset(inputs) == multiset(sorted)

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that implements the merge sort algorithm to sort an array of strings in
↪ lexicographic order.

The signature should be:

`p_14_4_merge_sort_lexicographic_seq(inputs: seq<string>) returns (sorted: seq<string>)`

The method should respect the following contract:

`ensures forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j], ensures multiset`
↪ `(inputs) == multiset(sorted)`

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnkQPziL7JmpZzumNMG4ybv6rvk2z

Model: gpt-5.5-2026-04-23

Created at: 1780748901

Finish reason: stop

Usage: 338 tokens in, and 5479 tokens out

```
{
  if |inputs| <= 1 {
    sorted := inputs;
    return;
  }

  var mid := |inputs| / 2;
  assert 0 < mid < |inputs|;

  var left := p_14_4_merge_sort_lexicographic_seq(inputs[..mid]);
  var right := p_14_4_merge_sort_lexicographic_seq(inputs[mid..]);

  sorted := [];
  var i := 0;
  var j := 0;

  while i < |left| || j < |right|
    invariant 0 <= i <= |left|
    invariant 0 <= j <= |right|
```

```

invariant |sorted| == i + j
invariant forall a, b :: 0 <= a < b < |sorted| ==> sorted[a] <= sorted[b]
invariant multiset(sorted) == multiset(left[..i]) + multiset(right[..j])
invariant forall a :: 0 <= a < |sorted| && i < |left| ==> sorted[a] <= left[i]
invariant forall a :: 0 <= a < |sorted| && j < |right| ==> sorted[a] <= right[j]
decreases |left| + |right| - i - j
{
  if i < |left| && (j == |right| || left[i] <= right[j]) {
    var oldSorted := sorted;
    var oldI := i;
    sorted := oldSorted + [left[oldI]];
    i := oldI + 1;

    assert forall a, b :: 0 <= a < b < |sorted| ==> sorted[a] <= sorted[b];
    assert left[..i] == left[..oldI] + [left[oldI]];
    assert multiset(sorted) == multiset(left[..i]) + multiset(right[..j]);

    assert forall a :: 0 <= a < |sorted| && i < |left| ==> sorted[a] <= left[i];
    assert forall a :: 0 <= a < |sorted| && j < |right| ==> sorted[a] <= right[j];
  } else {
    assert j < |right|;
    var oldSorted := sorted;
    var oldJ := j;
    sorted := oldSorted + [right[oldJ]];
    j := oldJ + 1;

    assert forall a, b :: 0 <= a < b < |sorted| ==> sorted[a] <= sorted[b];
    assert right[..j] == right[..oldJ] + [right[oldJ]];
    assert multiset(sorted) == multiset(left[..i]) + multiset(right[..j]);

    assert forall a :: 0 <= a < |sorted| && i < |left| ==> sorted[a] <= left[i];
    assert forall a :: 0 <= a < |sorted| && j < |right| ==> sorted[a] <= right[j];
  }
}

assert i == |left|;
assert j == |right|;
assert left[..i] == left;
assert right[..j] == right;
assert multiset(sorted) == multiset(left) + multiset(right);
assert inputs == inputs[..mid] + inputs[mid..];
assert multiset(inputs) == multiset(inputs[..mid]) + multiset(inputs[mid..]);
assert multiset(left) == multiset(inputs[..mid]);
assert multiset(right) == multiset(inputs[mid..]);
assert multiset(inputs) == multiset(sorted);
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(53,6): Error: assertion
  ↪ might not hold
|
53 | assert forall a :: 0 <= a < |sorted| && i < |left| ==> sorted[a] <= left[i];
| ~~~~~

```

Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
 ↪ Dafny body, including the curly braces that surround it. Do not show the signature
 ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnkSZzRFJT7otCuYoG2LUszvqkReC

Model: gpt-5.5-2026-04-23

Created at: 1780749035

Finish reason: stop

Usage: 1313 tokens in, and 5464 tokens out

```

{
  if |inputs| <= 1 {
    sorted := inputs;
    return;
  }

  var mid := |inputs| / 2;
  assert 0 < mid < |inputs|;

  var left := p_14_4_merge_sort_lexicographic_seq(inputs[..mid]);
  var right := p_14_4_merge_sort_lexicographic_seq(inputs[mid..]);

  sorted := [];
  var i := 0;
  var j := 0;

  while i < |left| || j < |right|
    invariant 0 <= i <= |left|
    invariant 0 <= j <= |right|
    invariant |sorted| == i + j
    invariant forall a, b :: 0 <= a < b < |sorted| ==> sorted[a] <= sorted[b]
    invariant multiset(sorted) == multiset(left[..i]) + multiset(right[..j])
    invariant forall a :: 0 <= a < |sorted| && i < |left| ==> sorted[a] <= left[i]
    invariant forall a :: 0 <= a < |sorted| && j < |right| ==> sorted[a] <= right[j]
    decreases |left| + |right| - i - j
  {
    if i == |left| {
      assert j < |right|;
      var oldSorted := sorted;

```

```

var oldJ := j;
assert forall a :: 0 <= a < |oldSorted| ==> oldSorted[a] <= right[oldJ];

sorted := oldSorted + [right[oldJ]];
j := oldJ + 1;

assert forall a, b | 0 <= a < b < |sorted| :: sorted[a] <= sorted[b] by {
  if b < |oldSorted| {
    assert sorted[a] == oldSorted[a];
    assert sorted[b] == oldSorted[b];
  } else {
    assert b == |oldSorted|;
    assert a < |oldSorted|;
    assert sorted[a] == oldSorted[a];
    assert sorted[b] == right[oldJ];
  }
}
assert right[..j] == right[..oldJ] + [right[oldJ]];
assert multiset(sorted) == multiset(left[..i]) + multiset(right[..j]);
assert forall a :: 0 <= a < |sorted| && i < |left| ==> sorted[a] <= left[i];
assert forall a | 0 <= a < |sorted| && j < |right| :: sorted[a] <= right[j] by {
  assert oldJ + 1 < |right|;
  assert right[oldJ] <= right[j];
  if a < |oldSorted| {
    assert sorted[a] == oldSorted[a];
    assert oldSorted[a] <= right[oldJ];
  } else {
    assert a == |oldSorted|;
    assert sorted[a] == right[oldJ];
  }
}
} else if j == |right| {
  assert i < |left|;
  var oldSorted := sorted;
  var oldI := i;
  assert forall a :: 0 <= a < |oldSorted| ==> oldSorted[a] <= left[oldI];

  sorted := oldSorted + [left[oldI]];
  i := oldI + 1;

  assert forall a, b | 0 <= a < b < |sorted| :: sorted[a] <= sorted[b] by {
    if b < |oldSorted| {
      assert sorted[a] == oldSorted[a];
      assert sorted[b] == oldSorted[b];
    } else {
      assert b == |oldSorted|;
      assert a < |oldSorted|;
      assert sorted[a] == oldSorted[a];
      assert sorted[b] == left[oldI];
    }
  }
}

```

```

assert left[..i] == left[..oldI] + [left[oldI]];
assert multiset(sorted) == multiset(left[..i]) + multiset(right[..j]);
assert forall a | 0 <= a < |sorted| && i < |left| :: sorted[a] <= left[i] by {
  assert oldI + 1 < |left|;
  assert left[oldI] <= left[i];
  if a < |oldSorted| {
    assert sorted[a] == oldSorted[a];
    assert oldSorted[a] <= left[oldI];
  } else {
    assert a == |oldSorted|;
    assert sorted[a] == left[oldI];
  }
}
}
assert forall a :: 0 <= a < |sorted| && j < |right| ==> sorted[a] <= right[j];
} else if left[i] <= right[j] {
  assert i < |left| && j < |right|;
  var oldSorted := sorted;
  var oldI := i;
  var oldJ := j;
  assert forall a :: 0 <= a < |oldSorted| ==> oldSorted[a] <= left[oldI];
  assert forall a :: 0 <= a < |oldSorted| ==> oldSorted[a] <= right[oldJ];
  assert left[oldI] <= right[oldJ];

  sorted := oldSorted + [left[oldI]];
  i := oldI + 1;

  assert forall a, b | 0 <= a < b < |sorted| :: sorted[a] <= sorted[b] by {
    if b < |oldSorted| {
      assert sorted[a] == oldSorted[a];
      assert sorted[b] == oldSorted[b];
    } else {
      assert b == |oldSorted|;
      assert a < |oldSorted|;
      assert sorted[a] == oldSorted[a];
      assert sorted[b] == left[oldI];
    }
  }
}
assert left[..i] == left[..oldI] + [left[oldI]];
assert multiset(sorted) == multiset(left[..i]) + multiset(right[..j]);
assert forall a | 0 <= a < |sorted| && i < |left| :: sorted[a] <= left[i] by {
  assert oldI + 1 < |left|;
  assert left[oldI] <= left[i];
  if a < |oldSorted| {
    assert sorted[a] == oldSorted[a];
    assert oldSorted[a] <= left[oldI];
  } else {
    assert a == |oldSorted|;
    assert sorted[a] == left[oldI];
  }
}
}
assert forall a | 0 <= a < |sorted| && j < |right| :: sorted[a] <= right[j] by {

```

```

    assert j == oldJ;
    if a < |oldSorted| {
      assert sorted[a] == oldSorted[a];
      assert oldSorted[a] <= right[oldJ];
    } else {
      assert a == |oldSorted|;
      assert sorted[a] == left[oldI];
      assert left[oldI] <= right[oldJ];
    }
  }
} else {
  assert i < |left| && j < |right|;
  assert right[j] <= left[i];
  var oldSorted := sorted;
  var oldI := i;
  var oldJ := j;
  assert forall a :: 0 <= a < |oldSorted| ==> oldSorted[a] <= left[oldI];
  assert forall a :: 0 <= a < |oldSorted| ==> oldSorted[a] <= right[oldJ];
  assert right[oldJ] <= left[oldI];

  sorted := oldSorted + [right[oldJ]];
  j := oldJ + 1;

  assert forall a, b | 0 <= a < b < |sorted| :: sorted[a] <= sorted[b] by {
    if b < |oldSorted| {
      assert sorted[a] == oldSorted[a];
      assert sorted[b] == oldSorted[b];
    } else {
      assert b == |oldSorted|;
      assert a < |oldSorted|;
      assert sorted[a] == oldSorted[a];
      assert sorted[b] == right[oldJ];
    }
  }
  assert right[..j] == right[..oldJ] + [right[oldJ]];
  assert multiset(sorted) == multiset(left[..i]) + multiset(right[..j]);
  assert forall a | 0 <= a < |sorted| && i < |left| :: sorted[a] <= left[i] by {
    assert i == oldI;
    if a < |oldSorted| {
      assert sorted[a] == oldSorted[a];
      assert oldSorted[a] <= left[oldI];
    } else {
      assert a == |oldSorted|;
      assert sorted[a] == right[oldJ];
      assert right[oldJ] <= left[oldI];
    }
  }
}
assert forall a | 0 <= a < |sorted| && j < |right| :: sorted[a] <= right[j] by {
  assert oldJ + 1 < |right|;
  assert right[oldJ] <= right[j];
  if a < |oldSorted| {

```

```

        assert sorted[a] == oldSorted[a];
        assert oldSorted[a] <= right[oldJ];
    } else {
        assert a == |oldSorted|;
        assert sorted[a] == right[oldJ];
    }
}
}
}

assert i == |left|;
assert j == |right|;
assert left[..i] == left;
assert right[..j] == right;
assert multiset(sorted) == multiset(left) + multiset(right);
assert inputs == inputs[..mid] + inputs[mid..];
assert multiset(inputs) == multiset(inputs[..mid]) + multiset(inputs[mid..]);
assert multiset(left) == multiset(inputs[..mid]);
assert multiset(right) == multiset(inputs[mid..]);
assert multiset(inputs) == multiset(sorted);
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(40,11): Error: unresolved
  ↪ identifier: b
  |
40 | if b < |oldSorted| {
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(41,24): Error: unresolved
  ↪ identifier: a
  |
41 | assert sorted[a] == oldSorted[a];
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(41,40): Error: unresolved
  ↪ identifier: a
  |
41 | assert sorted[a] == oldSorted[a];
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(42,24): Error: unresolved
  ↪ identifier: b
  |
42 | assert sorted[b] == oldSorted[b];
  | ^

```



```

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(42,40): Error: unresolved
  ↳ identifier: b
  |
42 | assert sorted[b] == oldSorted[b];
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(44,17): Error: unresolved
  ↳ identifier: b
  |
44 | assert b == |oldSorted|;
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(45,17): Error: unresolved
  ↳ identifier: a
  |
45 | assert a < |oldSorted|;
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(46,24): Error: unresolved
  ↳ identifier: a
  |
46 | assert sorted[a] == oldSorted[a];
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(46,40): Error: unresolved
  ↳ identifier: a
  |
46 | assert sorted[a] == oldSorted[a];
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(47,24): Error: unresolved
  ↳ identifier: b
  |
47 | assert sorted[b] == right[oldJ];
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(56,11): Error: unresolved
  ↳ identifier: a
  |
56 | if a < |oldSorted| {
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(57,24): Error: unresolved
  ↳ identifier: a
  |
57 | assert sorted[a] == oldSorted[a];
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(57,40): Error: unresolved
  ↳ identifier: a
  |

```

```

57 | assert sorted[a] == oldSorted[a];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(58,27): Error: unresolved
    ↪ identifier: a
    |
58 | assert oldSorted[a] <= right[oldJ];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(60,17): Error: unresolved
    ↪ identifier: a
    |
60 | assert a == |oldSorted|;
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(61,24): Error: unresolved
    ↪ identifier: a
    |
61 | assert sorted[a] == right[oldJ];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(74,11): Error: unresolved
    ↪ identifier: b
    |
74 | if b < |oldSorted| {
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(75,24): Error: unresolved
    ↪ identifier: a
    |
75 | assert sorted[a] == oldSorted[a];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(75,40): Error: unresolved
    ↪ identifier: a
    |
75 | assert sorted[a] == oldSorted[a];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(76,24): Error: unresolved
    ↪ identifier: b
    |
76 | assert sorted[b] == oldSorted[b];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(76,40): Error: unresolved
    ↪ identifier: b
    |
76 | assert sorted[b] == oldSorted[b];
    | ^

```

```

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(78,17): Error: unresolved
  ↳ identifier: b
  |
78 | assert b == |oldSorted|;
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(79,17): Error: unresolved
  ↳ identifier: a
  |
79 | assert a < |oldSorted|;
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(80,24): Error: unresolved
  ↳ identifier: a
  |
80 | assert sorted[a] == oldSorted[a];
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(80,40): Error: unresolved
  ↳ identifier: a
  |
80 | assert sorted[a] == oldSorted[a];
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(81,24): Error: unresolved
  ↳ identifier: b
  |
81 | assert sorted[b] == left[oldI];
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(89,11): Error: unresolved
  ↳ identifier: a
  |
89 | if a < |oldSorted| {
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(90,24): Error: unresolved
  ↳ identifier: a
  |
90 | assert sorted[a] == oldSorted[a];
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(90,40): Error: unresolved
  ↳ identifier: a
  |
90 | assert sorted[a] == oldSorted[a];
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(91,27): Error: unresolved
  ↳ identifier: a
  |

```

```

91 | assert oldSorted[a] <= left[oldI];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(93,17): Error: unresolved
    ↪ identifier: a
    |
93 | assert a == |oldSorted|;
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(94,24): Error: unresolved
    ↪ identifier: a
    |
94 | assert sorted[a] == left[oldI];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(111,11): Error: unresolved
    ↪ identifier: b
    |
111 | if b < |oldSorted| {
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(112,24): Error: unresolved
    ↪ identifier: a
    |
112 | assert sorted[a] == oldSorted[a];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(112,40): Error: unresolved
    ↪ identifier: a
    |
112 | assert sorted[a] == oldSorted[a];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(113,24): Error: unresolved
    ↪ identifier: b
    |
113 | assert sorted[b] == oldSorted[b];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(113,40): Error: unresolved
    ↪ identifier: b
    |
113 | assert sorted[b] == oldSorted[b];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(115,17): Error: unresolved
    ↪ identifier: b
    |
115 | assert b == |oldSorted|;
    | ^

```

```

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(116,17): Error: unresolved
  ↳ identifier: a
  |
116 | assert a < |oldSorted|;
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(117,24): Error: unresolved
  ↳ identifier: a
  |
117 | assert sorted[a] == oldSorted[a];
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(117,40): Error: unresolved
  ↳ identifier: a
  |
117 | assert sorted[a] == oldSorted[a];
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(118,24): Error: unresolved
  ↳ identifier: b
  |
118 | assert sorted[b] == left[oldI];
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(126,11): Error: unresolved
  ↳ identifier: a
  |
126 | if a < |oldSorted| {
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(127,24): Error: unresolved
  ↳ identifier: a
  |
127 | assert sorted[a] == oldSorted[a];
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(127,40): Error: unresolved
  ↳ identifier: a
  |
127 | assert sorted[a] == oldSorted[a];
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(128,27): Error: unresolved
  ↳ identifier: a
  |
128 | assert oldSorted[a] <= left[oldI];
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(130,17): Error: unresolved
  ↳ identifier: a
  |

```

```

130 | assert a == |oldSorted|;
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(131,24): Error: unresolved
  ↪ identifier: a
    |
131 | assert sorted[a] == left[oldI];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(136,11): Error: unresolved
  ↪ identifier: a
    |
136 | if a < |oldSorted| {
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(137,24): Error: unresolved
  ↪ identifier: a
    |
137 | assert sorted[a] == oldSorted[a];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(137,40): Error: unresolved
  ↪ identifier: a
    |
137 | assert sorted[a] == oldSorted[a];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(138,27): Error: unresolved
  ↪ identifier: a
    |
138 | assert oldSorted[a] <= right[oldJ];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(140,17): Error: unresolved
  ↪ identifier: a
    |
140 | assert a == |oldSorted|;
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(141,24): Error: unresolved
  ↪ identifier: a
    |
141 | assert sorted[a] == left[oldI];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(159,11): Error: unresolved
  ↪ identifier: b
    |
159 | if b < |oldSorted| {
    | ^

```

```

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(160,24): Error: unresolved
  ↳ identifier: a
  |
160 | assert sorted[a] == oldSorted[a];
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(160,40): Error: unresolved
  ↳ identifier: a
  |
160 | assert sorted[a] == oldSorted[a];
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(161,24): Error: unresolved
  ↳ identifier: b
  |
161 | assert sorted[b] == oldSorted[b];
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(161,40): Error: unresolved
  ↳ identifier: b
  |
161 | assert sorted[b] == oldSorted[b];
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(163,17): Error: unresolved
  ↳ identifier: b
  |
163 | assert b == |oldSorted|;
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(164,17): Error: unresolved
  ↳ identifier: a
  |
164 | assert a < |oldSorted|;
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(165,24): Error: unresolved
  ↳ identifier: a
  |
165 | assert sorted[a] == oldSorted[a];
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(165,40): Error: unresolved
  ↳ identifier: a
  |
165 | assert sorted[a] == oldSorted[a];
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(166,24): Error: unresolved
  ↳ identifier: b
  |

```

```

166 | assert sorted[b] == right[oldJ];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(173,11): Error: unresolved
  ↪ identifier: a
    |
173 | if a < |oldSorted| {
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(174,24): Error: unresolved
  ↪ identifier: a
    |
174 | assert sorted[a] == oldSorted[a];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(174,40): Error: unresolved
  ↪ identifier: a
    |
174 | assert sorted[a] == oldSorted[a];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(175,27): Error: unresolved
  ↪ identifier: a
    |
175 | assert oldSorted[a] <= left[oldI];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(177,17): Error: unresolved
  ↪ identifier: a
    |
177 | assert a == |oldSorted|;
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(178,24): Error: unresolved
  ↪ identifier: a
    |
178 | assert sorted[a] == right[oldJ];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(185,11): Error: unresolved
  ↪ identifier: a
    |
185 | if a < |oldSorted| {
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(186,24): Error: unresolved
  ↪ identifier: a
    |
186 | assert sorted[a] == oldSorted[a];
    | ^

```



```

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(186,40): Error: unresolved
  ↪ identifier: a
  |
186 | assert sorted[a] == oldSorted[a];
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(187,27): Error: unresolved
  ↪ identifier: a
  |
187 | assert oldSorted[a] <= right[oldJ];
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(189,17): Error: unresolved
  ↪ identifier: a
  |
189 | assert a == |oldSorted|;
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(190,24): Error: unresolved
  ↪ identifier: a
  |
190 | assert sorted[a] == right[oldJ];
  | ^

76 resolution/type errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
  ↪ Dafny body, including the curly braces that surround it. Do not show the signature
  ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

```

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnkUEJBVnUygN40A8lOXc1RvW2SEd

Model: gpt-5.5-2026-04-23

Created at: 1780749138

Finish reason: stop

Usage: 10955 tokens in, and 4192 tokens out

```

{
  if |inputs| <= 1 {
    sorted := inputs;
    return;
  }

  var mid := |inputs| / 2;
  assert 0 < mid < |inputs|;
  assert |inputs[..mid]| < |inputs|;
  assert |inputs[mid..]| < |inputs|;

  var left := p_14_4_merge_sort_lexicographic_seq(inputs[..mid]);

```

```

var right := p_14_4_merge_sort_lexicographic_seq(inputs[mid..]);

sorted := [];
var i := 0;
var j := 0;

while i < |left| || j < |right|
  invariant 0 <= i <= |left|
  invariant 0 <= j <= |right|
  invariant |sorted| == i + j
  invariant forall a, b :: 0 <= a < b < |sorted| ==> sorted[a] <= sorted[b]
  invariant multiset(sorted) == multiset(left[..i]) + multiset(right[..j])
  invariant forall a :: 0 <= a < |sorted| && i < |left| ==> sorted[a] <= left[i]
  invariant forall a :: 0 <= a < |sorted| && j < |right| ==> sorted[a] <= right[j]
  decreases |left| + |right| - i - j
{
  if i == |left| {
    assert j < |right|;
    var oldSorted := sorted;
    var oldJ := j;
    assert forall a, b :: 0 <= a < b < |oldSorted| ==> oldSorted[a] <= oldSorted[b];
    assert forall a :: 0 <= a < |oldSorted| ==> oldSorted[a] <= right[oldJ];

    sorted := oldSorted + [right[oldJ]];
    j := oldJ + 1;

    forall a, b | 0 <= a < b < |sorted|
      ensures sorted[a] <= sorted[b]
    {
      if b < |oldSorted| {
        assert sorted[a] == oldSorted[a];
        assert sorted[b] == oldSorted[b];
        assert oldSorted[a] <= oldSorted[b];
      } else {
        assert b == |oldSorted|;
        assert a < |oldSorted|;
        assert sorted[a] == oldSorted[a];
        assert sorted[b] == right[oldJ];
        assert oldSorted[a] <= right[oldJ];
      }
    }
  }
  assert right[..j] == right[..oldJ] + [right[oldJ]];
  assert multiset(sorted) == multiset(left[..i]) + multiset(right[..j]);
  assert forall a :: 0 <= a < |sorted| && i < |left| ==> sorted[a] <= left[i];
  forall a | 0 <= a < |sorted| && j < |right|
    ensures sorted[a] <= right[j]
  {
    assert oldJ + 1 < |right|;
    assert right[oldJ] <= right[j];
    if a < |oldSorted| {
      assert sorted[a] == oldSorted[a];
    }
  }
}

```

```

    assert oldSorted[a] <= right[oldJ];
  } else {
    assert a == |oldSorted|;
    assert sorted[a] == right[oldJ];
  }
}
} else if j == |right| {
  assert i < |left|;
  var oldSorted := sorted;
  var oldI := i;
  assert forall a, b :: 0 <= a < b < |oldSorted| ==> oldSorted[a] <= oldSorted[b];
  assert forall a :: 0 <= a < |oldSorted| ==> oldSorted[a] <= left[oldI];

  sorted := oldSorted + [left[oldI]];
  i := oldI + 1;

  forall a, b | 0 <= a < b < |sorted|
    ensures sorted[a] <= sorted[b]
  {
    if b < |oldSorted| {
      assert sorted[a] == oldSorted[a];
      assert sorted[b] == oldSorted[b];
      assert oldSorted[a] <= oldSorted[b];
    } else {
      assert b == |oldSorted|;
      assert a < |oldSorted|;
      assert sorted[a] == oldSorted[a];
      assert sorted[b] == left[oldI];
      assert oldSorted[a] <= left[oldI];
    }
  }
  assert left[..i] == left[..oldI] + [left[oldI]];
  assert multiset(sorted) == multiset(left[..i]) + multiset(right[..j]);
  forall a | 0 <= a < |sorted| && i < |left|
    ensures sorted[a] <= left[i]
  {
    assert oldI + 1 < |left|;
    assert left[oldI] <= left[i];
    if a < |oldSorted| {
      assert sorted[a] == oldSorted[a];
      assert oldSorted[a] <= left[oldI];
    } else {
      assert a == |oldSorted|;
      assert sorted[a] == left[oldI];
    }
  }
}
assert forall a :: 0 <= a < |sorted| && j < |right| ==> sorted[a] <= right[j];
} else if left[i] <= right[j] {
  assert i < |left| && j < |right|;
  var oldSorted := sorted;
  var oldI := i;

```

```

var oldJ := j;
assert forall a, b :: 0 <= a < b < |oldSorted| ==> oldSorted[a] <= oldSorted[b];
assert forall a :: 0 <= a < |oldSorted| ==> oldSorted[a] <= left[oldI];
assert forall a :: 0 <= a < |oldSorted| ==> oldSorted[a] <= right[oldJ];
assert left[oldI] <= right[oldJ];

sorted := oldSorted + [left[oldI]];
i := oldI + 1;

forall a, b | 0 <= a < b < |sorted|
  ensures sorted[a] <= sorted[b]
{
  if b < |oldSorted| {
    assert sorted[a] == oldSorted[a];
    assert sorted[b] == oldSorted[b];
    assert oldSorted[a] <= oldSorted[b];
  } else {
    assert b == |oldSorted|;
    assert a < |oldSorted|;
    assert sorted[a] == oldSorted[a];
    assert sorted[b] == left[oldI];
    assert oldSorted[a] <= left[oldI];
  }
}
assert left[..i] == left[..oldI] + [left[oldI]];
assert multiset(sorted) == multiset(left[..i]) + multiset(right[..j]);
forall a | 0 <= a < |sorted| && i < |left|
  ensures sorted[a] <= left[i]
{
  assert oldI + 1 < |left|;
  assert left[oldI] <= left[i];
  if a < |oldSorted| {
    assert sorted[a] == oldSorted[a];
    assert oldSorted[a] <= left[oldI];
  } else {
    assert a == |oldSorted|;
    assert sorted[a] == left[oldI];
  }
}
forall a | 0 <= a < |sorted| && j < |right|
  ensures sorted[a] <= right[j]
{
  assert j == oldJ;
  if a < |oldSorted| {
    assert sorted[a] == oldSorted[a];
    assert oldSorted[a] <= right[oldJ];
  } else {
    assert a == |oldSorted|;
    assert sorted[a] == left[oldI];
    assert left[oldI] <= right[oldJ];
  }
}

```

```

}
} else {
  assert i < |left| && j < |right|;
  assert right[j] <= left[i];
  var oldSorted := sorted;
  var oldI := i;
  var oldJ := j;
  assert forall a, b :: 0 <= a < b < |oldSorted| ==> oldSorted[a] <= oldSorted[b];
  assert forall a :: 0 <= a < |oldSorted| ==> oldSorted[a] <= left[oldI];
  assert forall a :: 0 <= a < |oldSorted| ==> oldSorted[a] <= right[oldJ];
  assert right[oldJ] <= left[oldI];

  sorted := oldSorted + [right[oldJ]];
  j := oldJ + 1;

  forall a, b | 0 <= a < b < |sorted|
    ensures sorted[a] <= sorted[b]
  {
    if b < |oldSorted| {
      assert sorted[a] == oldSorted[a];
      assert sorted[b] == oldSorted[b];
      assert oldSorted[a] <= oldSorted[b];
    } else {
      assert b == |oldSorted|;
      assert a < |oldSorted|;
      assert sorted[a] == oldSorted[a];
      assert sorted[b] == right[oldJ];
      assert oldSorted[a] <= right[oldJ];
    }
  }
  assert right[..j] == right[..oldJ] + [right[oldJ]];
  assert multiset(sorted) == multiset(left[..i]) + multiset(right[..j]);
  forall a | 0 <= a < |sorted| && i < |left|
    ensures sorted[a] <= left[i]
  {
    assert i == oldI;
    if a < |oldSorted| {
      assert sorted[a] == oldSorted[a];
      assert oldSorted[a] <= left[oldI];
    } else {
      assert a == |oldSorted|;
      assert sorted[a] == right[oldJ];
      assert right[oldJ] <= left[oldI];
    }
  }
}
forall a | 0 <= a < |sorted| && j < |right|
  ensures sorted[a] <= right[j]
{
  assert oldJ + 1 < |right|;
  assert right[oldJ] <= right[j];
  if a < |oldSorted| {

```

```

        assert sorted[a] == oldSorted[a];
        assert oldSorted[a] <= right[oldJ];
    } else {
        assert a == |oldSorted|;
        assert sorted[a] == right[oldJ];
    }
}
}
}

assert i == |left|;
assert j == |right|;
assert left[..i] == left;
assert right[..j] == right;
assert multiset(sorted) == multiset(left) + multiset(right);
assert inputs == inputs[..mid] + inputs[mid..];
assert multiset(inputs) == multiset(inputs[..mid]) + multiset(inputs[mid..]);
assert multiset(left) == multiset(inputs[..mid]);
assert multiset(right) == multiset(inputs[mid..]);
assert multiset(inputs) == multiset(sorted);
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(172,6): Error: assertion
  ↪ might not hold
|
172 | assert right[j] <= left[i];
    | ~~~~~

```

Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
 ↪ Dafny body, including the curly braces that surround it. Do not show the signature
 ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnkVRejfWfhP7DCZc5twxuiqMWZ2K

Model: gpt-5.5-2026-04-23

Created at: 1780749213

Finish reason: length

Usage: 10155 tokens in, and 2259 tokens out

The below is a raw response from GenAI that could not be parsed. It is likely that GenAI got 'stuck' in a loop and hit the pre-set token limit (i.e., finish reason is 'length').

Final Program

```
method p_14_4_merge_sort_lexicographic_seq(inputs: seq<string>) returns (sorted: seq<
  ↪ string>)
  ensures forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
  ensures multiset(inputs) == multiset(sorted)
{
  if |inputs| <= 1 {
    sorted := inputs;
    return;
  }

  var mid := |inputs| / 2;
  assert 0 < mid < |inputs|;
  assert |inputs[..mid]| < |inputs|;
  assert |inputs[mid..]| < |inputs|;

  var left := p_14_4_merge_sort_lexicographic_seq(inputs[..mid]);
  var right := p_14_4_merge_sort_lexicographic_seq(inputs[mid..]);

  sorted := [];
  var i := 0;
  var j := 0;

  while i < |left| || j < |right|
    invariant 0 <= i <= |left|
    invariant 0 <= j <= |right|
    invariant |sorted| == i + j
    invariant forall a, b :: 0 <= a < b < |sorted| ==> sorted[a] <= sorted[b]
    invariant multiset(sorted) == multiset(left[..i]) + multiset(right[..j])
    invariant forall a :: 0 <= a < |sorted| && i < |left| ==> sorted[a] <= left[i]
    invariant forall a :: 0 <= a < |sorted| && j < |right| ==> sorted[a] <= right[j]
    decreases |left| + |right| - i - j
  {
    if i == |left| {
      assert j < |right|;
      var oldSorted := sorted;
      var oldJ := j;
      assert forall a, b :: 0 <= a < b < |oldSorted| ==> oldSorted[a] <= oldSorted[b];
      assert forall a :: 0 <= a < |oldSorted| ==> oldSorted[a] <= right[oldJ];

      sorted := oldSorted + [right[oldJ]];
      j := oldJ + 1;

      forall a, b | 0 <= a < b < |sorted|
        ensures sorted[a] <= sorted[b]
      {
        if b < |oldSorted| {
          assert sorted[a] == oldSorted[a];
          assert sorted[b] == oldSorted[b];
          assert oldSorted[a] <= oldSorted[b];
        } else {
```

```

    assert b == |oldSorted|;
    assert a < |oldSorted|;
    assert sorted[a] == oldSorted[a];
    assert sorted[b] == right[oldJ];
    assert oldSorted[a] <= right[oldJ];
  }
}
assert right[..j] == right[..oldJ] + [right[oldJ]];
assert multiset(sorted) == multiset(left[..i]) + multiset(right[..j]);
assert forall a :: 0 <= a < |sorted| && i < |left| ==> sorted[a] <= left[i];
forall a | 0 <= a < |sorted| && j < |right|
  ensures sorted[a] <= right[j]
{
  assert oldJ + 1 < |right|;
  assert right[oldJ] <= right[j];
  if a < |oldSorted| {
    assert sorted[a] == oldSorted[a];
    assert oldSorted[a] <= right[oldJ];
  } else {
    assert a == |oldSorted|;
    assert sorted[a] == right[oldJ];
  }
}
}
} else if j == |right| {
  assert i < |left|;
  var oldSorted := sorted;
  var oldI := i;
  assert forall a, b :: 0 <= a < b < |oldSorted| ==> oldSorted[a] <= oldSorted[b];
  assert forall a :: 0 <= a < |oldSorted| ==> oldSorted[a] <= left[oldI];

  sorted := oldSorted + [left[oldI]];
  i := oldI + 1;

  forall a, b | 0 <= a < b < |sorted|
    ensures sorted[a] <= sorted[b]
  {
    if b < |oldSorted| {
      assert sorted[a] == oldSorted[a];
      assert sorted[b] == oldSorted[b];
      assert oldSorted[a] <= oldSorted[b];
    } else {
      assert b == |oldSorted|;
      assert a < |oldSorted|;
      assert sorted[a] == oldSorted[a];
      assert sorted[b] == left[oldI];
      assert oldSorted[a] <= left[oldI];
    }
  }
}
assert left[..i] == left[..oldI] + [left[oldI]];
assert multiset(sorted) == multiset(left[..i]) + multiset(right[..j]);
forall a | 0 <= a < |sorted| && i < |left|

```



```

    ensures sorted[a] <= left[i]
  {
    assert oldI + 1 < |left|;
    assert left[oldI] <= left[i];
    if a < |oldSorted| {
      assert sorted[a] == oldSorted[a];
      assert oldSorted[a] <= left[oldI];
    } else {
      assert a == |oldSorted|;
      assert sorted[a] == left[oldI];
    }
  }
}
assert forall a :: 0 <= a < |sorted| && j < |right| ==> sorted[a] <= right[j];
} else if left[i] <= right[j] {
  assert i < |left| && j < |right|;
  var oldSorted := sorted;
  var oldI := i;
  var oldJ := j;
  assert forall a, b :: 0 <= a < b < |oldSorted| ==> oldSorted[a] <= oldSorted[b];
  assert forall a :: 0 <= a < |oldSorted| ==> oldSorted[a] <= left[oldI];
  assert forall a :: 0 <= a < |oldSorted| ==> oldSorted[a] <= right[oldJ];
  assert left[oldI] <= right[oldJ];

  sorted := oldSorted + [left[oldI]];
  i := oldI + 1;

  forall a, b | 0 <= a < b < |sorted|
    ensures sorted[a] <= sorted[b]
  {
    if b < |oldSorted| {
      assert sorted[a] == oldSorted[a];
      assert sorted[b] == oldSorted[b];
      assert oldSorted[a] <= oldSorted[b];
    } else {
      assert b == |oldSorted|;
      assert a < |oldSorted|;
      assert sorted[a] == oldSorted[a];
      assert sorted[b] == left[oldI];
      assert oldSorted[a] <= left[oldI];
    }
  }
}
assert left[..i] == left[..oldI] + [left[oldI]];
assert multiset(sorted) == multiset(left[..i]) + multiset(right[..j]);
forall a | 0 <= a < |sorted| && i < |left|
  ensures sorted[a] <= left[i]
{
  assert oldI + 1 < |left|;
  assert left[oldI] <= left[i];
  if a < |oldSorted| {
    assert sorted[a] == oldSorted[a];
    assert oldSorted[a] <= left[oldI];
  }
}

```

```

    } else {
        assert a == |oldSorted|;
        assert sorted[a] == left[oldI];
    }
}
forall a | 0 <= a < |sorted| && j < |right|
    ensures sorted[a] <= right[j]
{
    assert j == oldJ;
    if a < |oldSorted| {
        assert sorted[a] == oldSorted[a];
        assert oldSorted[a] <= right[oldJ];
    } else {
        assert a == |oldSorted|;
        assert sorted[a] == left[oldI];
        assert left[oldI] <= right[oldJ];
    }
}
} else {
    assert i < |left| && j < |right|;
    assert right[j] <= left[i];
    var oldSorted := sorted;
    var oldI := i;
    var oldJ := j;
    assert forall a, b :: 0 <= a < b < |oldSorted| ==> oldSorted[a] <= oldSorted[b];
    assert forall a :: 0 <= a < |oldSorted| ==> oldSorted[a] <= left[oldI];
    assert forall a :: 0 <= a < |oldSorted| ==> oldSorted[a] <= right[oldJ];
    assert right[oldJ] <= left[oldI];

    sorted := oldSorted + [right[oldJ]];
    j := oldJ + 1;

    forall a, b | 0 <= a < b < |sorted|
        ensures sorted[a] <= sorted[b]
    {
        if b < |oldSorted| {
            assert sorted[a] == oldSorted[a];
            assert sorted[b] == oldSorted[b];
            assert oldSorted[a] <= oldSorted[b];
        } else {
            assert b == |oldSorted|;
            assert a < |oldSorted|;
            assert sorted[a] == oldSorted[a];
            assert sorted[b] == right[oldJ];
            assert oldSorted[a] <= right[oldJ];
        }
    }
}
assert right[..j] == right[..oldJ] + [right[oldJ]];
assert multiset(sorted) == multiset(left[..i]) + multiset(right[..j]);
forall a | 0 <= a < |sorted| && i < |left|
    ensures sorted[a] <= left[i]

```

```

{
  assert i == oldI;
  if a < |oldSorted| {
    assert sorted[a] == oldSorted[a];
    assert oldSorted[a] <= left[oldI];
  } else {
    assert a == |oldSorted|;
    assert sorted[a] == right[oldJ];
    assert right[oldJ] <= left[oldI];
  }
}
forall a | 0 <= a < |sorted| && j < |right|
  ensures sorted[a] <= right[j]
{
  assert oldJ + 1 < |right|;
  assert right[oldJ] <= right[j];
  if a < |oldSorted| {
    assert sorted[a] == oldSorted[a];
    assert oldSorted[a] <= right[oldJ];
  } else {
    assert a == |oldSorted|;
    assert sorted[a] == right[oldJ];
  }
}
}
}

assert i == |left|;
assert j == |right|;
assert left[..i] == left;
assert right[..j] == right;
assert multiset(sorted) == multiset(left) + multiset(right);
assert inputs == inputs[..mid] + inputs[mid..];
assert multiset(inputs) == multiset(inputs[..mid]) + multiset(inputs[mid..]);
assert multiset(left) == multiset(inputs[..mid]);
assert multiset(right) == multiset(inputs[mid..]);
assert multiset(inputs) == multiset(sorted);
}

```

Total Token Usage

Input tokens: 22761

Output tokens: 17394

Reasoning tokens: 11466

Sum of 'total tokens': 40155

Experiment Timings

Overall Experiment started at 1780748901557, ended at 1780749259063, lasting 357506ms (357.51 seconds)

Iteration #4 started at 1780749212929, ended at 1780749259063, lasting 46134ms (46.13 seconds)

Iteration #1 started at 1780748901575, ended at 1780749034871, lasting 133296ms (133.30 seconds)
Iteration #2 started at 1780749034871, ended at 1780749137961, lasting 103090ms (103.09 seconds)
Iteration #3 started at 1780749137961, ended at 1780749212929, lasting 74968ms (74.97 seconds)